Secret-derived constants on the GPU path

One attack/defense pair that ties the prototypes together

Confidential compiling stream — SPS Fellowship

Plan, not a result · `docs/research/gpu/narrative.agents.md`

Contents

1. Why GPU
2. The attack
3. The defense
4. How it fits, and the plan

Why GPU

The asset is on a GPU; our evidence is on a CPU

layer	CPU (today)	GPU
Silicon timing	ebpf-tracing	CUPTI / Nsight (stretch)
Dynamic taint	mlir_leak, leak_check	—
Codegen / artifact diff	leak_check (freezing, exfil)	this plan
Per-pass SMT	fcvd_ct	unchanged
2-safety BMC	proofs_l2_seabmc	+1 job
Static IFC	Staging_NI, initial	+1 rule

Both compiler-introduced findings in `leak_check` are explicitly scoped **away** from Triton/GPU. The pair below fills the codegen row on GPU and pulls the static, proof and regression rows along with it.

Why this pair and not another

Chosen: freezing on CUDA/ Triton

- Probe exists and is calibrated (3 guards)
- Constant folding runs on the FX graph **before** the C++/Triton split → sharp hypothesis
- `Staging_NI` already names the leak class

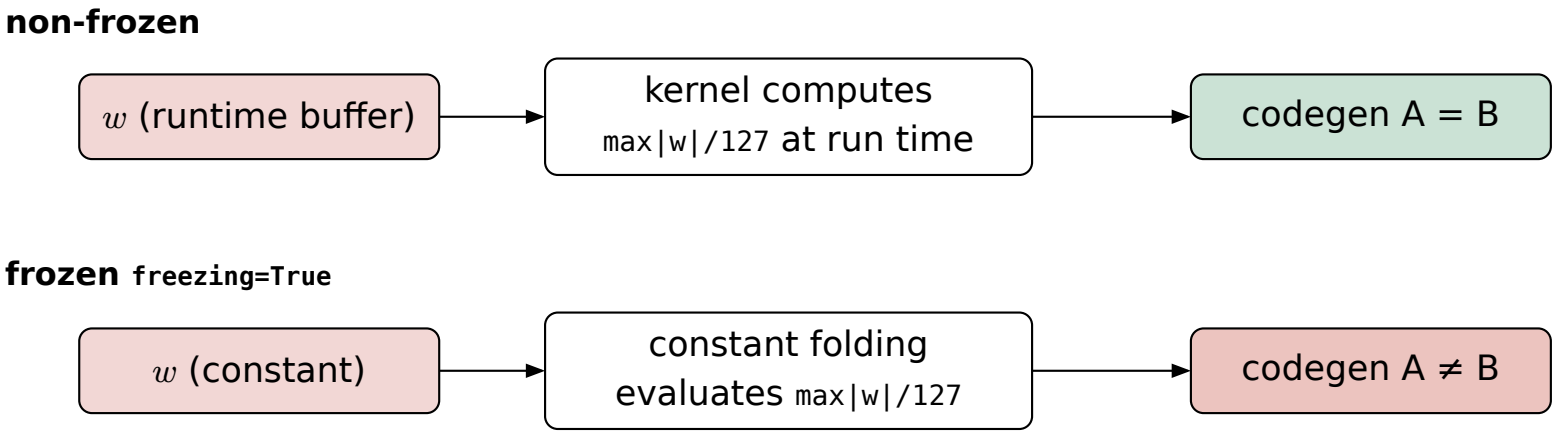
Deferred

- **LeftoverLocals on silicon**: NVIDIA was not affected; needs AMD/ Apple
- **MLIR→NVVM control-flow leaks**: Valgrind can’t see device code; rebuild the instrument

The attack

Freezing turns a weight into code (CPU)

measured

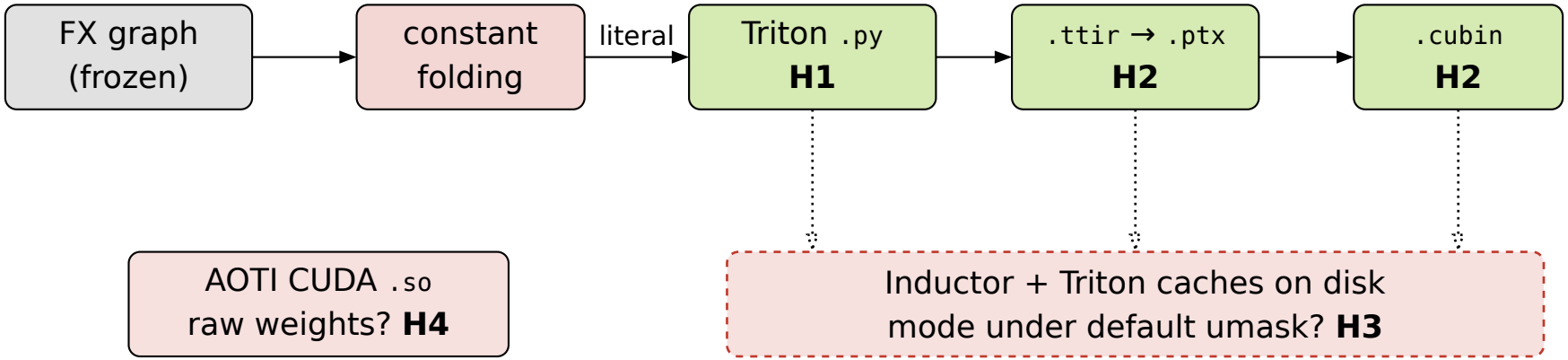



```
-   auto tmp1 = static_cast<float>(0.007874015718698502); // A
+   auto tmp1 = static_cast<float>(0.787401556968689);    // B
```

Fires on `quant_scale`, `wmax`, `wnorm`; `wsum` inconclusive. torch 2.13.0+cu130, Inductor CPU, x86-64.
`probe_freezing.py`

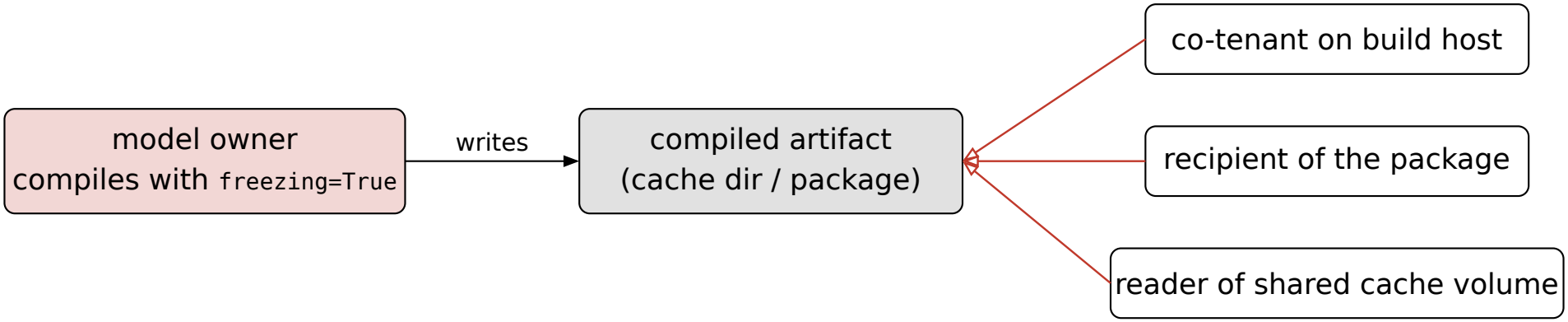
Where the literal could go on GPU

hypothesized



H3 is the composition: the freezing leak **and** the exfil leak in one artifact — a weight statistic on a shared filesystem, attacker never runs the model. **H5** unverified lead: IREE `const-eval` is the MLIR-native analogue.

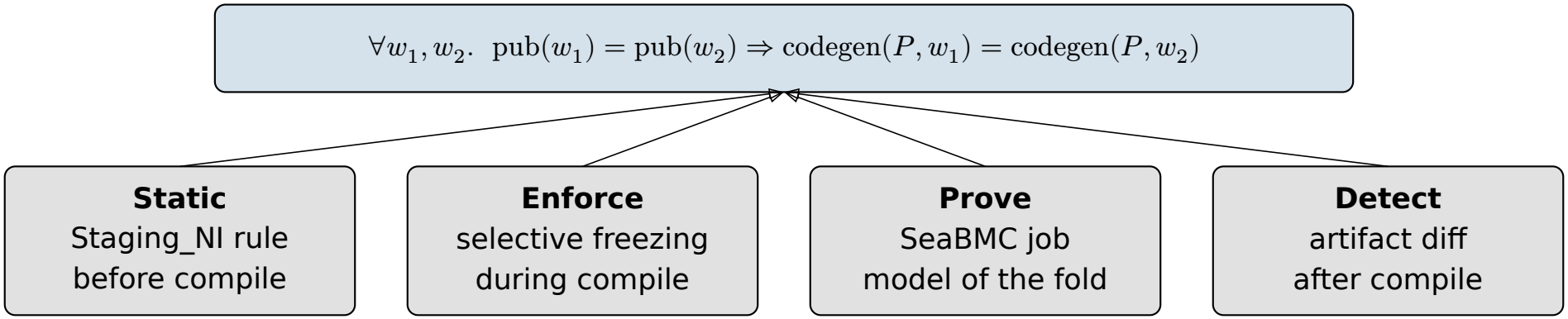
Who observes



- No timing, no execution of the victim model, no physical access.
- Ring from `threat-models.agents.md`: artifact reader.
- Property violated: **artifact non-interference** — codegen must not depend on protected weights.

The defense

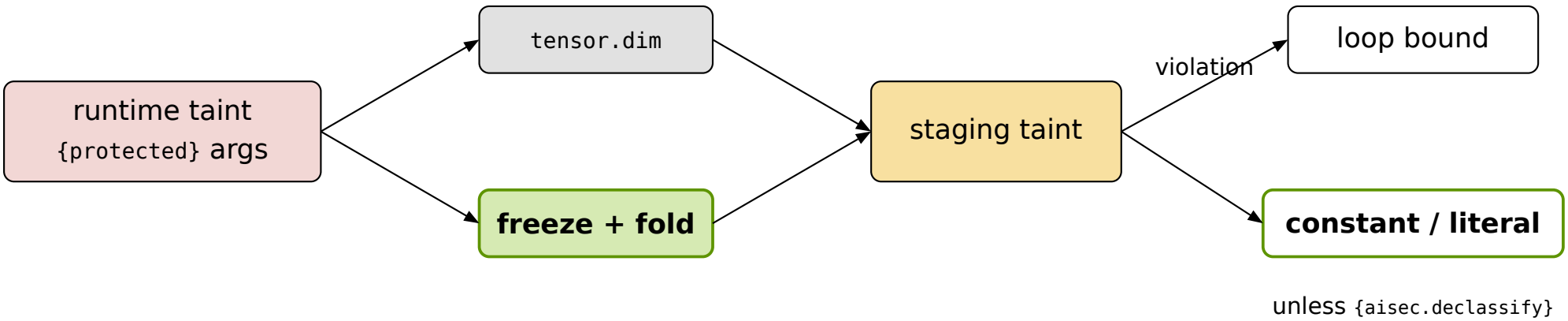
One property, four layers



Each layer extends an existing prototype; none is new infrastructure.

Static: freezing is runtime → staging

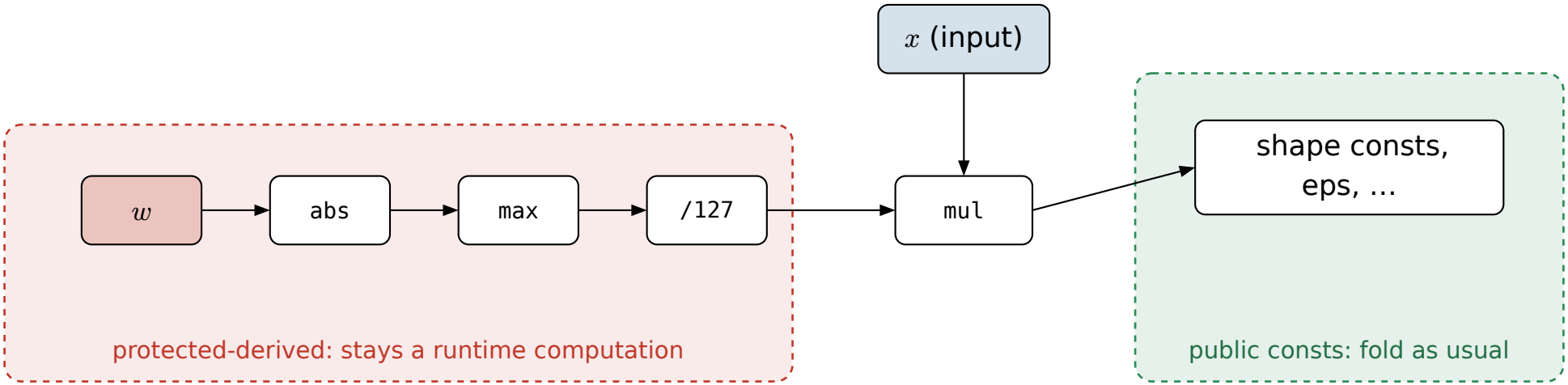
hypothesized



Grey = existing `Staging_NI` rule. Green = proposed. Apply at FX level first (the real Inductor path), then in Staging_NI on torch-mlir / IREE output. Calibrate: fires on the three measured surfaces, silent on non-frozen and declassified folds.

Enforce: freeze all but the protected subgraph

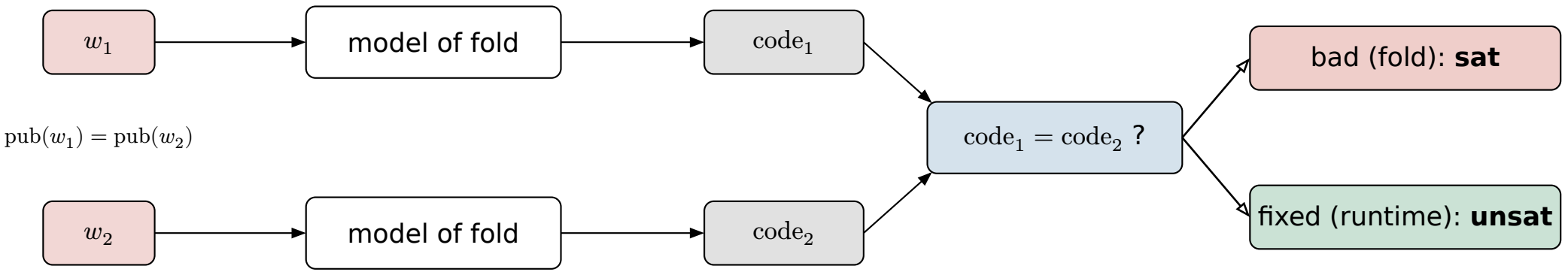
hypothesized



- Hook to exclude nodes from Inductor constant folding at 2.13: unverified lead. Fallback: pre-freeze FX rewrite that hides protected params behind an opaque op.
- Report latency: frozen vs selective vs non-frozen, same GPU.

Prove: the artifact is the observation

hypothesized



- One `add_l2_job` in `proofs_l2_seabmc`, same calibration rule as the seven existing pairs.
- A proof about a **model** of the fold, not about Inductor — stated that way.

Detect: artifact differential, as a regression fixture

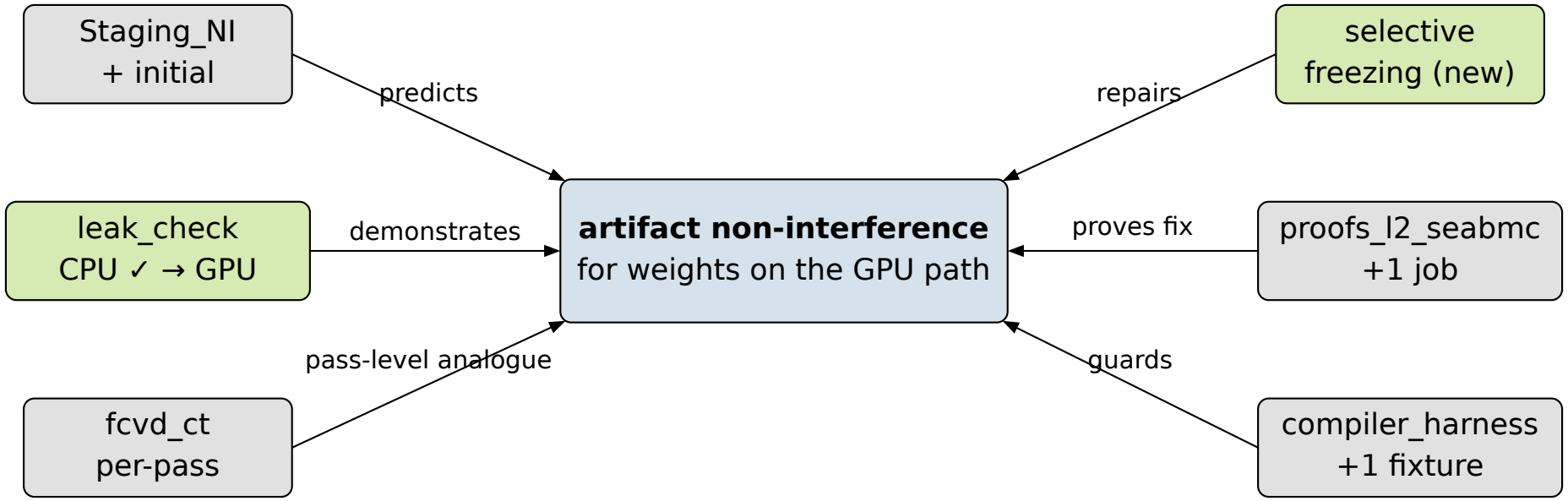
- Existing detector (`probe_autotune.normalize` + `kernel_body`), widened to every file a GPU compile writes: `.py` · `.ttir` · `.ttgir` · `.llir` · `.ptx` · `.cubin` (via `cuobjdump -sass`) · AOTI `.so`
- Plus a permission audit of every cache directory up to `/`.
- Same three guards: caches disabled per compile, same-class control A = A′, non-frozen A = B.

`compiler_harness`
fixture

`snapshot.yaml`
`bad/` freezing on
`fixed/` selective freezing
expected: bad A≠B, fixed A=B

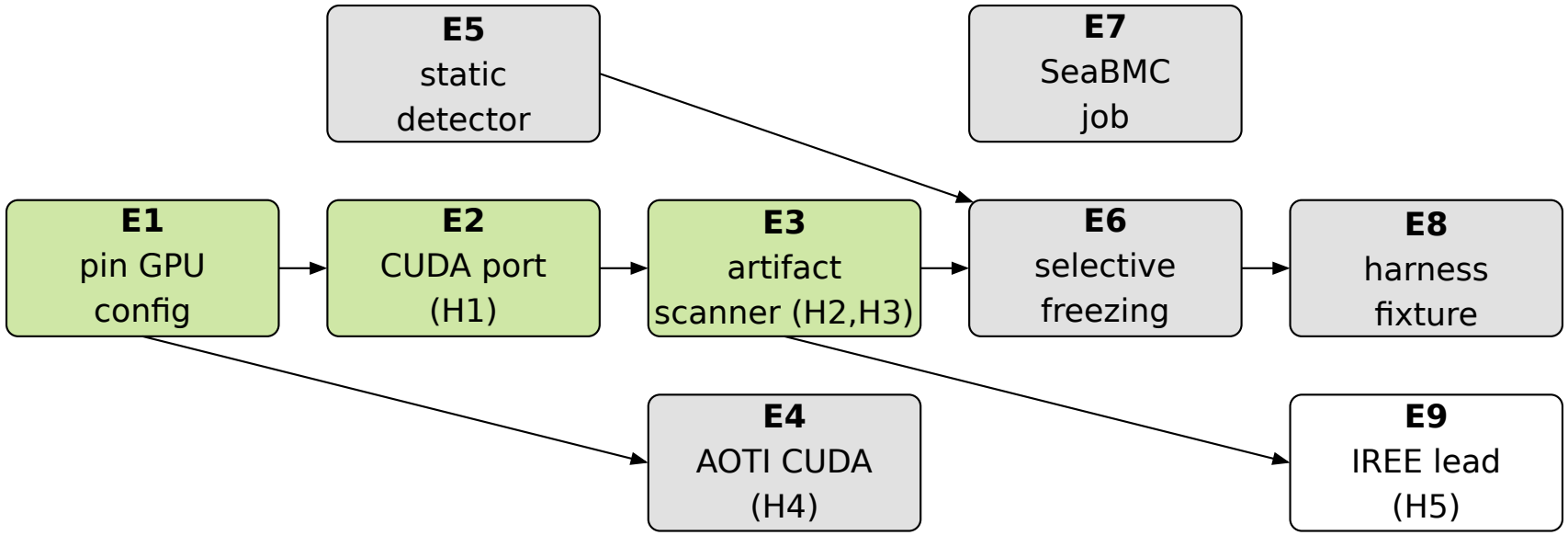
How it fits, and the plan

The prototypes, as one story



Out of scope for this pair: `ebpf-tracing`, `mlir_leak` (timing / control-flow channels; this leak is neither).

Work items



Green = minimum publishable unit. Natural owners: E2–E4 `leak_check`, E5 `Staging_NI`, E7 `proofs_l2_seabmc`. Hardware: one cloud GPU, one pinned SKU.

What would change the story

if	then
H1 false: Triton folds into a runtime buffer, not a literal	still reportable at the config point; switch to buffer-valued folding (freezing note §4), new detector
H3 false: literal in source, not on shared disk	observer narrows to package recipients; still a finding
selective freezing erases freezing’s speedup	defense becomes a policy (“don’t freeze protected models”), not a pass — report the numbers

SaTML deadline (2026-09-29) has passed: this is rebuttal / camera-ready / follow-up material — a GPU column for §3, artifact non-interference for §4.

Freezing turns a weight into code.

We predict it, show it on the GPU path, repair it, and prove the repair.